

LOOPS AND SETS

Do_While_Loops

🕒 Created time	August 28, 2026 8:19 AM
☰ Category	Loops_Sets
👤 Last edited by	 R.A R.A
🕒 Last updated time	August 28, 2026 11:03 AM

```
bool isValidAge;
int age;

int testNumber = 0;

do
{
    Console.Write(testNumber);
    testNumber += 3;
} while (testNumber < 10);

do
{
```

```

Console.Write("How old are you? ");
string ageText = Console.ReadLine();

isValidAge = int.TryParse(ageText, out age);

if (isValidAge == false)
{
    Console.WriteLine("That was an invalid age.");
}
} while (isValidAge == false);

Console.WriteLine($"Your age is {age}.");

do
{

} while (true);

while (true)
{

}

```

לולאת `do` ולולאת `while` הן למעשה כמעט אותו הדבר.

אבל יש ביניהן הבדל חשוב.

לולאות `do` ו-`while` ימשיכו לרוץ שוב ושוב עד שהתנאי הזה יהפוך ל-`false`:

```

~~~~~
do
{

} while (true);
~~~~~

```

כרגע התנאי מוגדר כ-`true`, מכיוון שזהו ערך ברירת המחדל הזמני שמוכנס לשם כאשר יוצרים את ה-snippet:

```
~~~~~
do
{

} while (true);

while (true)
{

}
~~~~~
```

זה קצת דומה ל-`if statement`, שבו כל מה שנמצא בתוך הסוגריים `()` חייב להיות ביטוי שמוערך ל-`true` או ל-`false`, כלומר `bool`. אבל מה שיקרה בשתי הלולאות האלה הוא שהקוד שנמצא בתוך הסוגריים המסולסלים `{ }` ימשיך לרוץ עד שהביטוי של ה-`while` יהפוך ל-`false`.

```
~~~~~
do
{

} while (true);
~~~~~
```

בעבר ביקשנו מהמשתמש להזין את הגיל שלו:

```
~~~~~
Console.Write("How old are you? ");
string ageText = Console.ReadLine();
~~~~~
```

ומשם היינו עושים:

```
~~~~~
bool isValidAge = int.TryParse(ageText, out int age);
```

והיינו אומרים: אוקיי, האם זה גיל תקין?

ואם לא, היינו עושים:

```
~~~~~
if (isValidAge == false)
{
    Console.WriteLine("That was an invalid age.");
    return;
}
~~~~~
```

אבל אולי אנחנו רוצים לתת למשתמש הזדמנות נוספת.

כי לפעמים קורות טעויות — מתחילים להקליד משהו, אולי האצבע בטעות לוחצת על מקש לא נכון, ולוחצים Enter לפני ששמים לב שבטעות נוספה למשל האות **K** בסוף הגיל. זה יכול לקרות.

אנחנו לא תמיד רוצים פשוט לומר למשתמש: אוקיי, סיימת, תחזור מאוחר יותר. במקום זה, אולי נרצה לומר: הגיל שהזנת לא היה תקין, נסה שוב — ולתת למשתמש הזדמנות להזין אותו מחדש.

אבל איך נעשה את זה בעזרת הדברים שלמדנו עד עכשיו? עם מה שלמדנו עד עכשיו, עדיין לא היינו יודעים איך לעשות את זה.

לכן נשתמש בלולאת **do**.

אנחנו נעטוף את כל הקוד הזה בתוך לולאת **do**:

```
~~~~~
Console.Write("How old are you? ");
string ageText = Console.ReadLine();

bool isValidAge = int.TryParse(ageText, out int age);
~~~~~
```

```
~~~~~
do
{
```

```

Console.Write("How old are you? ");
string ageText = Console.ReadLine();

bool isValidAge = int.TryParse(ageText, out int age);
} while (true);
~~~~~

```

עכשיו הקוד שלנו נמצא בתוך לולאת **do**.

אנחנו שואלים את המשתמש בן כמה הוא, ולאחר מכן בודקים האם הערך שהוא הזין תקין.

עכשיו הקוד שלנו נראה כך:

```

~~~~~
do
{
    Console.Write("How old are you? ");
    string ageText = Console.ReadLine();

    bool isValidAge = int.TryParse(ageText, out int age);
} while (true);

if (isValidAge == false)
{
    Console.WriteLine("That was an invalid age.");
    return;
}
~~~~~

```

עכשיו אנחנו יכולים לקחת את ה- **if statement** הזה:

```

~~~~~
if (isValidAge == false)
{
    Console.WriteLine("That was an invalid age.");
    return;
}
~~~~~

```

ולכניס גם אותו לתוך לולאת ה- **do**:

```

~~~~~
do
{
    Console.Write("How old are you? ");
    string ageText = Console.ReadLine();

    bool isValidAge = int.TryParse(ageText, out int age);

    if (isValidAge == false)
    {
        Console.WriteLine("That was an invalid age.");
        return;
    }
} while (true);
~~~~~

```

בכל מקרה זה יעבוד — בין אם ה־`if` נמצא בתוך לולאת ה־`do`, ובין אם הוא נשאר מחוץ ללולאה.

אם `isValidAge` הוא `false`, אנחנו אומרים למשתמש שהגיל שהזין אינו תקין. אבל במקום לעשות `return`, פשוט נסיר את ה־`return`:

```

~~~~~
do
{
    Console.Write("How old are you? ");
    string ageText = Console.ReadLine();

    bool isValidAge = int.TryParse(ageText, out int age);

    if (isValidAge == false)
    {
        Console.WriteLine("That was an invalid age.");
    }
} while (true);
~~~~~

```

עכשיו אנחנו יכולים לבדוק את `isValidAge`.

שימו לב שהקומפיילר אומר לנו שאנחנו לא יכולים לעשות את זה, משום שהכרזנו על `isValidAge` בתוך לולאת ה-`do`.

לכן עכשיו אנחנו צריכים להגדיר את `bool isValidAge` מעל לולאת ה-`do`, ובתוך הלולאה רק להציב בו ערך:

```
~~~~~
bool isValidAge;

do
{
    Console.WriteLine("How old are you? ");
    string ageText = Console.ReadLine();

    isValidAge = int.TryParse(ageText, out int age);

    if (isValidAge == false)
    {
        Console.WriteLine("That was an invalid age.");
    }
} while (isValidAge);
~~~~~
```

עכשיו אנחנו יכולים לבדוק אותו.

הלולאה תגיע ל-`do`, תריץ את כל הקוד שנמצא בתוך הסוגריים המסולסלים, ולאחר מכן תבדוק האם היא צריכה להמשיך ולחזור בחזרה להתחלה.
יש לנו:

```
while (isValidAge == false);
```

אם הזנו גיל לא תקין בתוך `ageText`, כך שלא ניתן היה לבצע לו `Parse`, אז מה שיקרה הוא שהקוד יאמר: אוקיי, זה לא גיל תקין.

לאחר מכן הוא יחזור להתחלה, יבקש שוב את הגיל, יקבל שוב את הטקסט וינסה לבצע לו `Parse`.

אם הפעם הגיל תקין, אנחנו נדלג על החלק הבא:

```
~~~~~
if (isValidAge == false)
```

```
{
    Console.WriteLine("That was an invalid age.");
}
```

~~~~~

מכיוון שהגיל תקין.

בנוסף, נצא מלולאת ה-`do` ונמשיך להריץ את שאר הקוד.

הלולאה תמשיך לרוץ שוב ושוב עד שהביטוי הבא יהיה `false`:

```
~~~~~
while (isValidAge == false);
~~~~~
```

זה יכול להיות מעט מבלבל.

`false` באמת שווה ל-`isValidAge` כאשר `true` יחזיר `isValidAge == false`.

כלומר, הלולאה אומרת: המשך לחזור להתחלה כל עוד הביטוי של `isValidAge == false` מוערך ל-`true`.

אם `isValidAge` הוא `false`, אז:

```
false == false
```

מחזיר `true`.

ולכן חוזרים לראש הלולאה.

זה ממשיך עד ש-`isValidAge` הופך ל-`true`.

ואז:

```
true == false
```

מחזיר `false`.

ולכן לולאת ה-`do/while` נעצרת.

```
~~~~~
bool isValidAge;

do
{
 Console.Write("How old are you? ");
 string ageText = Console.ReadLine();
```



```

 isValidAge = int.TryParse(ageText, out int age);

 if (isValidAge == false)
 {
 Console.WriteLine("That was an invalid age.");
 }
} while (isValidAge == false);
~~~~~

```

יש באג אחד בקוד הזה:

```

~~~~~
Console.WriteLine($"Your age is {age}.");
~~~~~

```

הקומפיילר אומר: לא, אי אפשר לעשות את זה.

והוא נותן את השגיאה:

```
The name 'age' does not exist in the current context
```

זהו מסר חשוב.

אנחנו יכולים להתווכח ולומר: אבל כן, `age` כן קיים!

אבל לא כדאי להתווכח עם הכלים — בדרך כלל הם חכמים יותר מאיתנו.

במקרה הזה הכרזנו על `age` בתוך ה-`scope` של לולאת ה-`do`.

לכן `age` קיים רק בתוך ה-`scope` הזה:

```

~~~~~
do
{
 Console.Write("How old are you? ");
 string ageText = Console.ReadLine();

 isValidAge = int.TryParse(ageText, out int age);

 if (isValidAge == false)
 {
 Console.WriteLine("That was an invalid age.");
 }
}

```

```
}
```

```
~~~~~
```

לכן אנחנו צריכים להגדיר את `int age` כאן למעלה:

```
~~~~~
```

```
bool isValidAge;
```

```
int age;
```

```
do
```

```
{
```

```
 Console.WriteLine("How old are you? ");
```

```
 string ageText = Console.ReadLine();
```

```
 isValidAge = int.TryParse(ageText, out int age);
```

```
 if (isValidAge == false)
```

```
 {
```

```
 Console.WriteLine("That was an invalid age.");
```

```
 }
```

```
} while (isValidAge == false);
```

```
Console.WriteLine($"Your age is {age}.");
```

```
~~~~~
```

ואז למחוק את המילה `int` מכאן:

```
~~~~~
```

```
isValidAge = int.TryParse(ageText, out int age);
```

```
~~~~~
```

כך:

```
~~~~~
```

```
isValidAge = int.TryParse(ageText, out age);
```

```
~~~~~
```

ה־ `TryParse` יכניס ערך לתוך `age`.

עכשיו נוכל להשתמש ב- `age` גם מחוץ ללולאת ה- `do`:

```
~~~~~  
bool isValidAge;
int age;

do
{
 Console.Write("How old are you? ");
 string ageText = Console.ReadLine();

 isValidAge = int.TryParse(ageText, out age);

 if (isValidAge == false)
 {
 Console.WriteLine("That was an invalid age.");
 }
} while (isValidAge == false);

Console.WriteLine($"Your age is {age}.");
~~~~~
```

עכשיו נריץ את הקוד:

```
How old are you? Eighteen  
compiler: "That was an invalid age."  
How old are you? Tim  
compiler: "That was an invalid age."  
How old are you? Bob  
compiler: "That was an invalid age."  
How old are you? Sue  
compiler: "That was an invalid age."  
How old are you? 18  
compiler: "Your age is 18"
```

כך עובדת לולאת `do`.

היא תבדוק את תנאי ה- `while` הבא:

```
~~~~~  
while (isValidAge == false);
```

```
~~~~~
```

ותמשיך לחזור להתחלה עד שהביטוי הזה יהיה `false`:

```
~~~~~
```

```
while (isValidAge == false);
```

```
~~~~~
```

יש כאן סכנה מסוימת:

```
~~~~~
```

```
int testNumber = 0;
```

```
do
```

```
{
```

```
 Console.Write(testNumber);
```

```
 testNumber += 3;
```

```
} while (testNumber != 10);
```

```
~~~~~
```

הקוד הזה מוסיף 3 לערך של `testNumber`.

זכרו ש-`+=` הוא בעצם דרך מקוצרת לכתוב:

```
~~~~~
```

```
testNumber = testNumber + 3;
```

```
~~~~~
```

אנחנו פשוט מקצרים את זה, מכיוון שהכתיבה המלאה מעט חוזרת על עצמה.

אז האם אתם רואים את הבעיה כאן?

```
~~~~~
```

```
int testNumber = 0;
```

```
do
```

```
{
```

```
 Console.Write(testNumber);
```

```
 testNumber += 3;
```

```
} while (testNumber != 10);
~~~~~
```

יש לנו כאן **לולאה אינסופית** שתמשיך לרוץ לנצח!

אבל למה?

הרי רצינו לעצור ב-10.

נכון — אבל אמרנו:

```
testNumber != 10
```

האם `testNumber` אי פעם יהיה שווה ל-10?

לא.

מפני שהתחלנו ב-0, ובכל פעם אנחנו מוסיפים 3:

```
3 / 6 / 9 / 12
```

אנחנו אף פעם לא מגיעים ל-10.

לזה קוראים **Infinite Loop — לולאה אינסופית**.

וזה דבר רע.

לולאה אינסופית פירושה שהאפליקציה שלנו עלולה להמשיך לרוץ ללא הפסקה, להגיע למצב שבו היא צורכת עוד ועוד משאבים, ולהוביל לבעיות שונות באפליקציה.

לכן אנחנו צריכים לוודא שהתנאים של הלולאות שלנו בנויים בצורה נכונה ומדויקת.

אם אנחנו רוצים לומר: עצור ב-10, אנחנו צריכים לומר ש-`testNumber` קטן מ-10:

```
~~~~~  
int testNumber = 0;

do
{
 Console.Write(testNumber);
 testNumber += 3;
} while (testNumber < 10);
~~~~~
```

במקרה הזה הלולאה תמשיך כך:

10 > 3 הוא `true` — חוזרים להתחלה.

`10 > 6` הוא `true` — חוזרים להתחלה.

`10 > 9` הוא `true` — חוזרים להתחלה.

`10 > 12` הוא `false`.

לכן יורדים לשורה הבאה והלולאה נעצרת.

צריך גם לזכור את האפשרות של מספרים שליליים.

לדוגמה, אם השתמשנו בטעות בסימן הלא נכון ועשינו `==` במקום `+=`, נקבל שוב לולאה אינסופית.

הסיבה היא שאנחנו אף פעם לא מוודאים שהמספר מתקדם בכיוון הנכון.

לכן חשוב מאוד שכאשר אנחנו עובדים עם לולאות, תמיד נוודא שללולאה שלנו קיימת דרך יציאה.

```
~~~~~  
int testNumber = 0;

do
{
 Console.Write(testNumber);
 testNumber -= 3;
} while (testNumber < 10);
~~~~~
```

דוגמה נוספת ללולאה אינסופית היא אם נכתוב:

```
while (isValidAge);
```

לדוגמה:

```
~~~~~  
do
{
 Console.Write("How old are you? ");
 string ageText = Console.ReadLine();

 isValidAge = int.TryParse(ageText, out age);

 if (isValidAge == false)
```

```

 {
 Console.WriteLine("That was an invalid age.");
 }
} while (isValidAge);

Console.WriteLine($"Your age is {age}.");
~~~~~

```

עכשיו נריץ את הקוד:

```

How old are you? 18
How old are you? 25
How old are you? 36
How old are you? 543
How old are you? 4
How old are you? 5

```

הוא לא מאפשר לנו לצאת מהלולאה!

אבל אם נכתוב:

```
"Test"
```

נקבל:

```

compiler: "That was an invalid age."
Your age is 0

```

ואז הלולאה מסתיימת.

זה לגמרי הפוך ממה שאנחנו רוצים!

לכן צריך להיות זהירים מאוד עם התנאי של הלולאה.

אבל לולאות הן כלי מאוד חזק במצבים כאלה.

אנחנו יכולים להמשיך לחזור למשתמש ולומר: לא! אני צריך ממך גיל תקין!

אולי גם נרצה לתת למשתמש דרך אחרת לצאת.

לדוגמה:

תן לי גיל תקין, או הקלד `exit` כדי לסיים את התוכנית.

יכול להיות שלמשתמש אין אפשרות או רצון להזין גיל תקין, ולכן צריך לתת לו דרך לצאת.

אנחנו לא רוצים להכריח את המשתמש להזין גיל מזויף, ולכן גם את זה צריך לקחת בחשבון.

אפשר להשתמש בלולאות גם לדברים אחרים.

לדוגמה, במקום לקבל `ageText`, אפשר לשאול:

```
do you want to keep going?
```

אם המשתמש אומר `yes`, אפשר לבדוק האם התשובה היא `yes`, אולי באמצעות `ToLower()`, ואם כן להריץ שוב את הלולאה.

אפשר גם לאסוף כך מספר אנשים.

לדוגמה, אם אנחנו מבקשים שמות של אנשים שמגיעים למסיבה:  
המשתמש מזין שם.

אנחנו שואלים האם מגיע אורח נוסף.

המשתמש אומר `yes`.

הלולאה חוזרת להתחלה.

אנחנו שוב שואלים מה שם האורח.

המשתמש אומר `Bob`.

שואלים האם יש אורח נוסף.

המשתמש אומר `yes`.

חוזרים שוב להתחלה.

שואלים מה השם.

המשתמש אומר `Sue`.

שואלים אם מגיע אורח נוסף.

המשתמש אומר `No`.

ואז יוצאים מהלולאה וממשיכים לשאר הקוד.

לכן לולאת `do` יכולה להיות מאוד שימושית וחזקה.

---

זכרו, קיימים שני סוגים שאנחנו מדברים עליהם כאן:

- לולאת `do`, כלומר `do/while`

- לולאת `while`

לולאת `while` כמעט זהה ללולאת `do`.

ההבדל הוא:



```
~~~~~
do
{

} while (true);

while (true)
{

}
~~~~~
```

בלולאת `do`, אנחנו קודם מגיעים ל- `do` ומריצים את הקוד:

```
~~~~~
do
{

} while (true);
~~~~~
```

ורק לאחר מכן בודקים אם התנאי הוא `true` או `false`:

```
~~~~~
while (true);
~~~~~
```

וכל עוד הוא `true`, חוזרים להתחלה.

לעומת זאת, בלולאת `while`:

```
~~~~~
while (true)
{

}
~~~~~
```

הבדיקה נמצאת בחלק העליון, ורק אחריה מגיעים הסוגריים המסולסלים.

כלומר:

```
~~~~~  
do
{
Run the code at least once .
} while (true);
~~~~~
```

לעומת:

```
~~~~~  
while (true)
{
Runs the code 0 or more times.
}
~~~~~
```

בלולאת **do**, הקוד תמיד ירוץ לפחות פעם אחת, ורק לאחר מכן נבדוק האם צריך להמשיך ולהריץ אותו שוב.

בלולאת **while**, קודם בודקים האם בכלל צריך להריץ את הקוד.

לאחר מכן הקוד ירוץ כמה פעמים שצריך.

לכן יכול להיות שבמקרה של לולאת **while**, הקוד אפילו לא ירוץ פעם אחת.

## מתי נשתמש ב־ **do/while**, ומתי ב־ **while** ?

במקרה שלנו, אנחנו מבקשים מהמשתמש להזין גיל ולאחר מכן חוזרים להתחלה אם הגיל אינו תקין.

זה אומר שאנחנו חייבים לשאול את המשתמש את הגיל שלו לפחות פעם אחת.

אנחנו לא יכולים לבדוק האם הגיל שלו תקין לפני שבכלל ביקשנו ממנו את הגיל.

לכן קודם נריץ את הקוד פעם אחת:

```
~~~~~  
do
{
```

```

Console.Write("How old are you? ");
string ageText = Console.ReadLine();

isValidAge = int.TryParse(ageText, out age);

if (isValidAge == false)
{
 Console.WriteLine("That was an invalid age.");
}
} while (isValidAge == false);
~~~~~

```

ולכן במקרה הזה אנחנו צריכים לולאת **do**.

יש מקרים אחרים שבהם אולי יש לנו משהו שצריך לנקות, או קוד כלשהו שייתכן שצריך לבצע עליו פעולת ניקוי.

אנחנו יכולים לבצע בדיקה ולשאול:

האם צריך לבצע ניקוי?

אם כן, נבצע את הניקוי כאן:

```

~~~~~
while (true)
{

}
~~~~~

```

אם לא — פשוט נמשיך הלאה.

ואם כן צריך לבצע ניקוי, נעשה את הניקוי ולאחר מכן נשאל:

האם סיימנו את הניקוי?

אם לא — נעשה אותו שוב.

ושוב.

ושוב.

עד שנסיים את הניקוי.

ברגע שסיימנו, נצא מהלולאה.

לכן הבחירה בין `do/while` לבין `while` תלויה בתרחיש שלנו.

אבל `do/while` ו-`while` הן בסופו של דבר כמעט אותו הדבר, מלבד ההבדל במספר הפעמים המינימלי שהקוד שלהן ירוץ.

**לולאת `while` יכולה לרוץ 0 פעמים.**

כלומר, ייתכן שהיא בכלל לא תרוץ.

**לולאת `do` תרוץ לפחות פעם אחת.**

זכרו: בלולאת `do`, הביטוי הבא הוא בדיקה בדיוק כמו תנאי של `if statement`:

```
~~~~~  
while (isValidAge == false);
~~~~~
```

ובתוך התנאי הזה אפשר להשתמש באותם דברים שאנחנו כבר מכירים, למשל:

`and`

`or`

`&&`

`||`

`ToLower()`

`<`

`>`

טווחי גיל וכדומה.